

MPCA - UNIT 1

µproc. vs µcontroller.

CISC vs RISC.

INSTRUCTION SET ARCHITECTURE : ARM

Modes: U FISH sADSc

User: 1 0 0 0 0

FIQ: 1 0 0 0 1

IRQ: 1 0 0 1 0

Supervisor: 1 0 0 1 1
SVC

About: 1 0 1 1 1
ABT

Undefined: 1 1 0 1 1
VBT

System: 1 1 1 1 1

r13 → SP
r14 → LR
r15 → PC

Banked registers (mode specific)

User: r13(SP), ~~r14(LR)~~ r14(LR)

FIQ: r8-r12, r13, r14, SP, SPSR

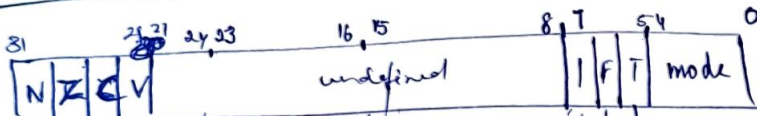
Everything else: r13, r14, SPSR, mode, abt.

(SPSR_irq, SPSR_SVC, SPSR_und etc)

high speed

System/abt

STATUS REGISTER



Overflow
Zero
Carry
negative

flags
f

status
s

extension
x

Thumb (16 bit) → code density
0: ARM
1: Thumb
RO-R7 barrel shifter X

FIQ disable
0: enabled
1: disabled

IRQ disable
0: enabled
1: disabled

control
c

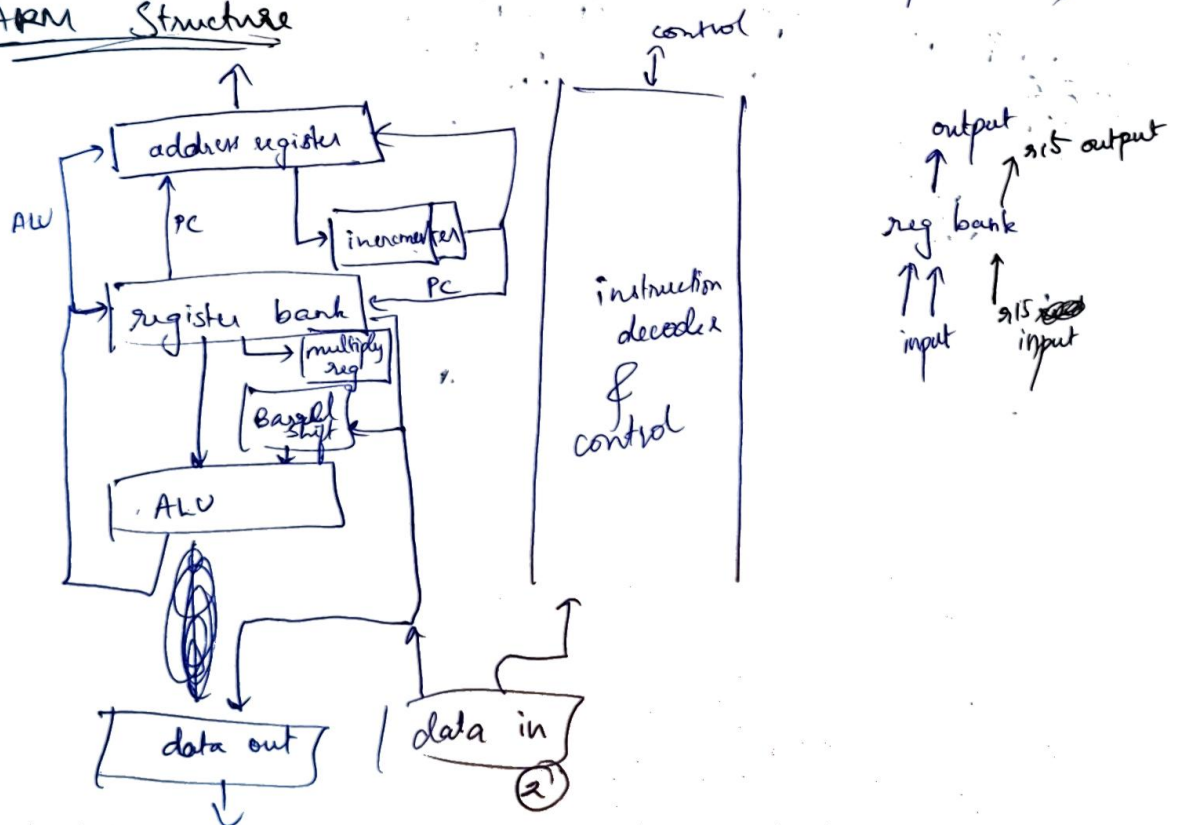
CPSR

SPSR

RISC Features Rejected by ARM

- ① Register windows
shift to new registers every process → too many regs needed
↓
large space on chip
- ② ~~Can~~ execute every ~~instr~~ instr. in one cycle.
Data processing ✓
Others X
Load / Store → 2 mem accesses.
- ③ Branches / delayed branches.
↳ mess with pipelining
branch after following exec
↳ does not scale to superscalar
branch prediction X.

ARM Structure



ARM COMMANDS : DATA PROCESSING

- MOV $R2, R1$
- MVN $R2, R1$ $\neg R1 \rightarrow R2$
- CMP $R1, R2$ $R1 - R2$, set cf
- CMN $R1, R2$ $R1 + R2$, set cf
- TST $R1, R2$ bitwise. $R1 \& R2$, set cf. check some particular bit
- TEQ $R1, R2$ bitwise equality. $R1 \oplus R2$, set cf

ADD - - - , SUB
ADC CBC

RSB $R1, R2, R3$
 $R3 - R2 \rightarrow R1$

~~RBC~~ RBC
 $R3 - R2 + C - 1 \rightarrow R1$

Bitwise logical

AND $R0, R1, R2$

OR $R0, R1, R2$

EOR $R0, R1, R2$ (XOR)

BIC $R0, R1, R2$ $\{ R1 \& \neg R2$

bit clear: clear all bits of $R1$ where $R2$ is one

Barrel shifter

LSC

LSR

~~ASR~~

ASR

ROR

RRX

Byte.
increment by 1
LDRB
STRB

Branches, control inst

Half words: increment by 2
LDRH
STRH

• DATA

A: HWORD $0x10, 0x20$

• TEXT

LDR $R1, =A$

LDRH $R2, [R1]$

ADD $R1, R1, \#4$ } increment twice

STRH $R2, [R1]$

Addressing modes

Preindexing: ~~LDR~~ LDR R2, [R0, R1] $\rightarrow$ R2 = mem[R0+R1]
R0 unchanged

Preindexing w/ writeback (autoindexing): LDR R2, [R0, R1]! $\rightarrow$ "

Postindexing: LDR R2, [R0], R1 $\rightarrow$ R2 = mem[R0]
R0 = R0 + R1

Multiply

32 bits:

MUL R0, R1, R2

$R0 = R1 \times R2$

MLA R0, R1, R2, R3

$R0 = (R1 \times R2) + R3$

64 bit:

(U/S)MULL R0, R1, R2, R3

R1:R0 = $R2 \times R3$

(U/S)MLAL R0, R1, R2, R3

R1:R0 = $(R2 \times R3) + R1:R0$

Matrix row major using MLA:

MLA Rd, Rr, Rn, Rc

$\begin{matrix} R_r & R_n & R_c \\ \downarrow & \downarrow & \downarrow \\ \text{row\#} & \text{no. of elements in each row} & \text{col\#} \end{matrix}$

$(\text{row\#} \times n) + \text{col\#}$

Multiple Load STM

LDM STM (IA/IB/DA/DB) Rsource{!} {RA-Rn}

Copy 512 bytes from A to B if data transfer rate is 32 bytes.

32 bytes/4 $\rightarrow$ 8 registers

512/32 $\rightarrow$ 16 transfers

LDR R9, =A

LDR R10, =B

MOV R11, #16

loop:

LDMIA R9! {R0-R7}

STMIA R10! {R0-R7}

CMP R11, #16

BNE loop

Stack operations		
mode	LDM	STM
EA	DA	IB
ED	IA	DB
FA	DB	IA
FD	IB	DA

pop (pointing to DA in STM column)
push (pointing to IB in LDM column)

Functions

BL name

return: MOV PC, LR (or) BX LR

Store into stack: STMFD R13!, {...}
(parameters)

load parameters: LDMFD R13!, {...}

nexted procedure call: load LR into stack and access

Software Interrupts

exec_{th} → supervisor mode

(after completing current insts)

SWI 0x00: display char

SWI 0x02 = display string

SWI 0x11 = terminate

SWI N → nth system routine

hardware interrupt: some external • hardware cond.

software interrupt: exception instruction

- trap: failure condition of processor.

→ also called Supervisor Call

MSR/MRS

MSR: Move to Status from Reg

MRS: " " Reg → Status.

~~MRS [CPSR/SPSR], Rn~~

MRS Rn, [CPSR/SPSR]

MSR CPSR-[field], Rn
SPSR-[field]

- F: flag (24:31)

~~- M: mode (8:15)~~

- X: extension (16:23)

- C: control (0:7)

- S: status (16:23)

(16:23)

Swap (mem loc ↔ reg)

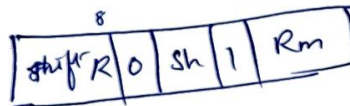
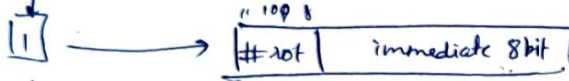
Swap Rd, Rd, [mem-addr]
↓ ↓
destination original
 (same)

INSTRUCTION ENCODING

1110 → Always

shift type
 00 → LL
 01 → LR
 10 → AR
 11 → ROR

Data processing



Branch

$$\text{Offset} = \frac{\text{Target Address} - (\text{PC} + 8)}{4}$$



1: BC
0: B

1110 1010 = EA

Multiplication

mul rd, rm, rs

mla rd, rm, rs, rn

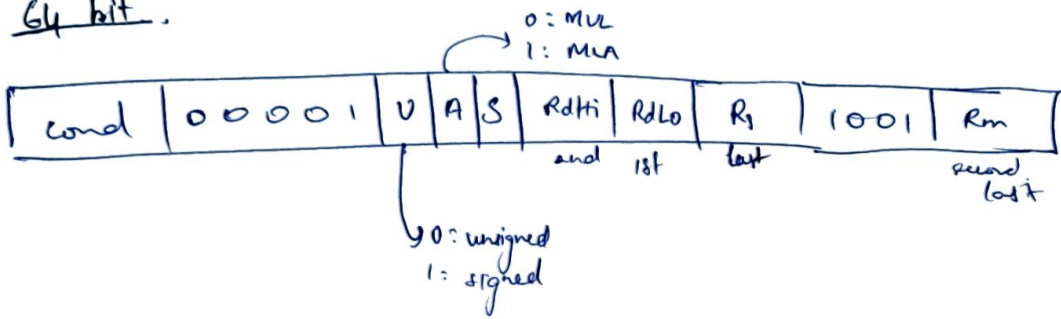
32 bits



0: MUL
1: MLA

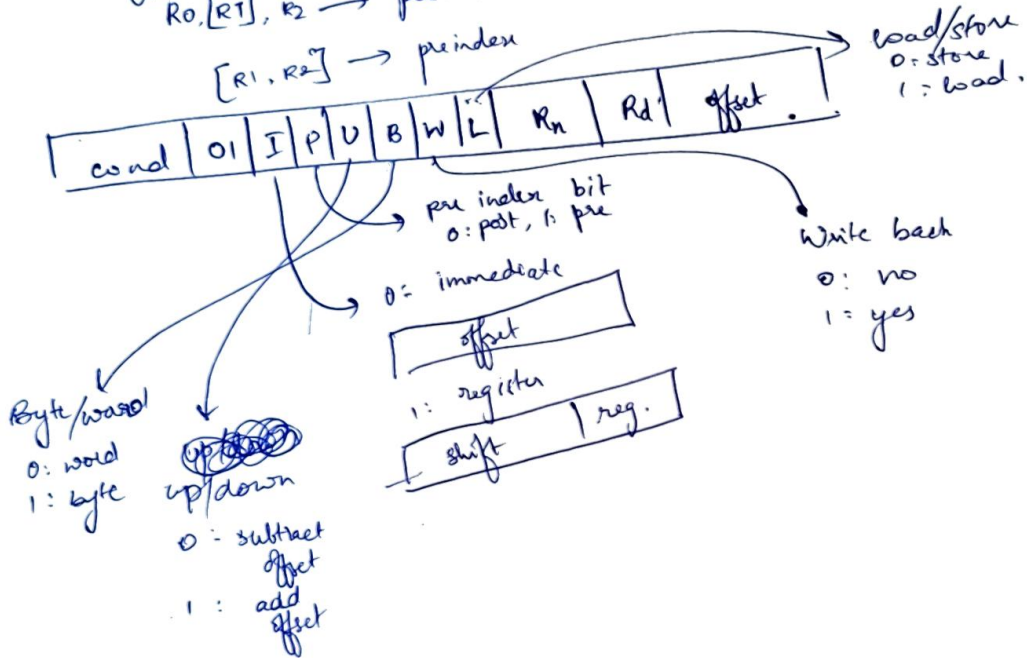
(6)

64 bit

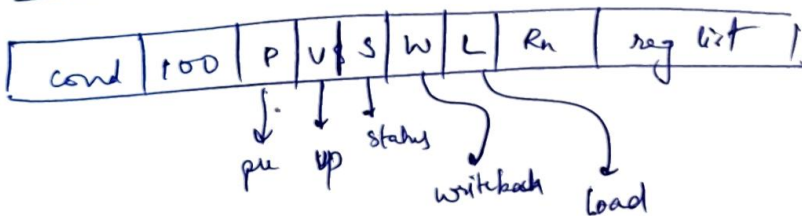


Data Transfer

No offset = pre index w/ no write back
 Ro, [R1], R2 → post index
 [R1, R2] → pre index



Block Transfer



Swap

